Rockchip IMU 开发指南

文件标识：RK-KF-YF-H58

发布版本：V1.0.0

日期：2026-04-29

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文介绍 Rockchip Linux 平台下 IMU（惯性测量单元，Inertial Measurement Unit）驱动的配置方法与基本调试方法。

产品版本

芯片名称	内核版本
ALL	Linux 6.1 / Linux 5.10

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	张裕阳	2026-04-29	初始版本

目录

Rockchip IMU 开发指南

1. Linux IIO 框架
    1.1 Linux IIO 框架概述
        1.1.1 IMU 系统层级图
        1.1.2 驱动层职责
    1.2 IIO 子系统关键组件
        1.2.1 IIO 设备
            1.2.1.1 IIO 设备通道
            1.2.1.2 IIO 设备属性
        1.2.2 IIO buffer
            1.2.2.1 buffer 配置
            1.2.2.2 scan element 配置
        1.2.3 IIO trigger
            1.2.3.1 trigger 回调
        1.2.4 triggered-buffer
    1.3 驱动适配 IIO 框架流程
        1.3.1 DTS & Kconfig
            1.3.1.1 总线配置
            1.3.1.2 中断 / 引脚配置
            1.3.1.3 Kconfig 配置
        1.3.2 驱动框架搭建
        1.3.3 定义 IIO 设备通道与属性
        1.3.4 芯片初始化实现
        1.3.5 配置 IIO trigger
        1.3.6 配置 triggered-buffer
2. IMU 使用指南
    2.1 IIO sysfs 配置
        2.1.1 采样频率
        2.1.2 量程与精度
        2.1.3 低通滤波
        2.1.4 中断使能
    2.2 数据读取与处理
        2.2.1 IIO sysfs read
        2.2.2 IIO buffer 读取流程
            2.2.2.1 确认设备节点路径
            2.2.2.2 理解帧布局
            2.2.2.3 读取 scale 值
            2.2.2.4 使能 buffer
            2.2.2.5 循环读取帧
            2.2.2.6 转换并打印数据
3. 数据分析与标定
    3.1 数据校准与标定
        3.1.1 了解标定参数
        3.1.2 安装标定工具
        3.1.3 采集标定数据
        3.1.4 创建配置文件
        3.1.5 执行标定
        3.1.6 将标定结果写入算法配置
    3.2 判断数据是否正常
        3.2.1 采集数据
        3.2.2 数据检查

4. 常见问题排查
    4.1 PCB 布板注意事项
    4.2 零偏与温漂是否需要工厂矫正
    4.3 内置 LPF 对数据的影响

1. Linux IIO 框架

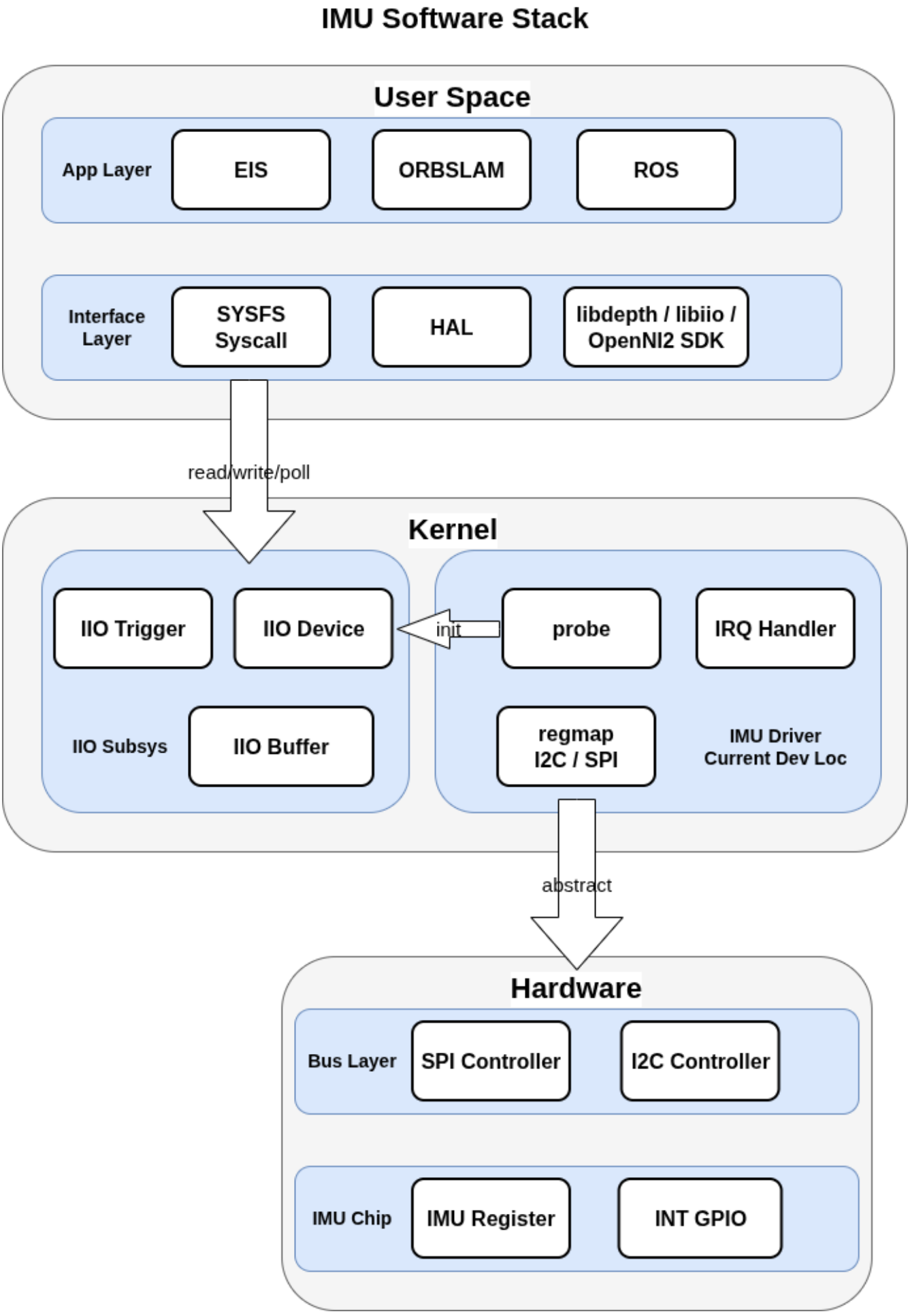
本章介绍 Linux IIO（Industrial I/O）子系统的整体架构与关键组件，并提供 IMU 芯片适配 IIO 框架的流程。本章内容与 IMU 芯片型号无关，适用于所有基于 IIO 框架的 IMU 驱动开发。

1.1 Linux IIO 框架概述

IIO 是 Linux 内核专门为 ADC、DAC、加速计、陀螺仪等模拟/混合信号器件设计的子系统。IIO 子系统既为编写多种嵌入式传感器的驱动程序提供了统一框架，也为访问这些传感器的用户空间应用程序提供了标准接口。

1.1.1 IMU 系统层级图

下图展示了 IMU 软件栈的完整层级。


IMU Software Stack
User Space
App Layer
EIS
ORBSLAM
ROS
Interface Layer
SYSFS Syscall
HAL
libdepth / libiio / OpenNI2 SDK
read/write/poll
Kernel
IIO Trigger
IIO Device
init
probe
IRQ Handler
IIO Subsys
IIO Buffer
regmap I2C / SPI
IMU Driver Current Dev Loc
abstract
Hardware
Bus Layer
SPI Controller
I2C Controller
IMU Chip
IMU Register
INT GPIO

1.1.2 驱动层职责

驱动层职责在于**向 IIO 框架注册设备、适配底层通信总线、配置 trigger 与 buffer**。

1.2 IIO 子系统关键组件

本节逐一说明 IIO 子系统各关键组件的作用与驱动侧实现。

组件	结构体	所在头文件	作用
IIO设备	`iio_dev`	`linux/iio/iio.h`	代表一个 IIO 设备
IIO设备通道	`iio_chan_spec`	`linux/iio/iio.h`	描述单个传感器通道
IIO设备属性	`iio_info`	`linux/iio/iio.h`	IIO属性及回调集合
IIO触发器	`iio_trigger`	`linux/iio/trigger.h`	传感器中采样触发源
IIO缓冲区	`iio_buffer`	`linux/iio/buffer.h`	记录传感器采样数据

1.2.1 IIO 设备

IIO 设备是硬件传感器的软件抽象。上层用户通过访问 IIO 设备的 sysfs 文件或字符设备文件实现对传感器的控制。驱动开发者需要通过 `iio_device_alloc` / `devm_iio_device_alloc` 分配 IIO 设备，并在 IIO 设备挂载 IIO 设备通道、IIO 设备属性等成员，最后交由 `iio_device_register` / `devm_iio_device_register` 注册到 IIO 框架。

1.2.1.1 IIO 设备通道

IIO 设备通道是对数据通道的一种抽象表示。一个 IIO 设备可以有一个或多个通道，例如一个加速度计最多可拥有三个通道，分别表示 X、Y、Z 轴上的加速度。IIO 通道由结构体 `iio_chan_spec` 描述。以加速计为例，其驱动程序需要按如下方式定义：

```
static const struct iio_chan_spec accel_x_channel[] = {
    {
        .type = IIO_ACCEL,              // 类型为加速计
        .channel2 = IIO_MOD_X,          // X轴修饰符
        .modified = 1,
        .info_mask_separate = BIT(IIO_CHAN_INFO_RAW),
        .info_mask_shared_by_type = BIT(IIO_CHAN_INFO_SCALE) |
BIT(IIO_CHAN_INFO_SAMP_FREQ),
    },
};
```

Linux 内核启动时，会在 `/sys/bus/iio/devices/iio:deviceX/` 目录下创建 channel 私有文件 `in_accel_x_raw` 与相同类型 channel 共享文件 `in_accel_scale` & `in_accel_sampling_frequency`，sysfs 文件命名格式为：

```
in_{chan_type}{channel && indexed}_{channel2 && modified}_{info_type}
```

1.2.1.2 IIO 设备属性

`iio_info` 定义了 IIO 设备的常量属性和信息。此处仅列举 IMU 驱动开发高相关的几项属性。

attrs

attrs 定义了 IIO 设备通用的某些属性。例如加速计、陀螺仪的可选量程、可选采样频率参数，传感器通用滤波器配置可通过 attrs 属性实现。

```
// 配置只读属性
static IIO_CONST_ATTR(in_accel_sampling_frequency_available,
                      "1 10 100 1000 10000 100000");

static struct attribute *xxx_attrs[] = {
    &iio_const_attr_in_accel_sampling_frequency_available.dev_attr.attr,
};
```

read_raw

read_raw 定义用户读取 IIO 数据通道 sysfs 文件的方法。

```
int (*read_raw)(struct iio_dev *indio_dev,
                struct iio_chan_spec const *chan,
                int *val, int *val2, long mask);
```

参数说明

- chan - 数据类型，如IIO_ACCEL / IIO_ANGL_VEL / IIO_TEMP
- mask - sysfs 文件属性，如IIO_CHAN_INFO_RAW / IIO_CHAN_INFO_SCALE / IIO_CHAN_INFO_SAMP_FREQ
- val - 整数读数
- val2 - 小数读数

`read_raw` 返回值与数据打印格式表如下：

返回值	数据打印格式
IIO_VAL_INT	val
IIO_VAL_INT_PLUS_MICRO	$val + val2 \times 10^{-6}$
IIO_VAL_INT_PLUS_NANO	$val + val2 \times 10^{-9}$

write_raw

write_raw 定义用户写 IIO 数据通道 sysfs 文件的方法。

```
int (*write_raw)(struct iio_dev *indio_dev,
                 struct iio_chan_spec const *chan,
                 int val, int val2, long mask);
```

读写前建议调用 `iio_device_claim_direct_mode` 进行并发访问保护，防止 buffer 运行期间并发修改寄存器引起数据错误。

1.2.2 IIO buffer

IIO 框架提供了缓冲区作为传感器数据临时存放的内存空间，底层基于`kfifo` 环形缓冲区实现。上层用户可以通过 `/dev/iio:deviceX` 字符设备节点一次性从缓冲区读取一帧或者多帧数据，从而降低CPU负载。驱动本身不需要管理缓冲区内存。

1.2.2.1 buffer 配置

驱动通过接口 `devm_iio_triggered_buffer_setup` 在 sysfs 目录下创建 buffer 目录，并提供以下可配置关键参数。

- `buffer/length`：配置的是该缓冲区能容纳的记录条数，实际占用内存 $≈ length × 单条记录长度$。默认值为2。
- `buffer/watermark`：配置内核累积 N 帧采样记录后向用户态发出可读事件的水线阈值。默认值为1。
- `buffer/enable`：使能 IIO buffer 采集，写1开启，写0关闭。使能后 IIO 框架自动回调 `set_trigger_state` 开启触发器。

1.2.2.2 scan element 配置

`scan element` 是存放在 `IIO buffer` 中的通道读数所关联的元信息。IIO 通道定义了数据通道以何种方式被写入 IIO buffer。以上述加速计为例，IIO 通道定义如下：

```
static const struct iio_chan_spec accel_x_channel[] = {
    {
        .scan_index = 0,            // 写入缓冲区的排列顺序
        .scan_type = {
            .sign = 's',            // 符号位
            .realbits = 16,         // 真实数据位数
            .storagebits = 16,      // 存储位数，需8bit对齐，未对齐补padding
            .endianness = IIO_BE,   // 存储字节序，请按约束配置。高位字节存放在低地址，需配置大端序
        },
    },
};
```

通道挂载后，内核初始化时会在 sysfs 目录 `/sys/bus/iio/devices/iio:deviceX/scan_elements/` 创建 `scan_element` 配置文件。

- `scan_elements/in_*_en`：用于启用通道。当且仅当其属性值为非零时，一次触发采集才会包含此通道的数据样本。
- `scan_elements/in_*_index`：`scan_index`决定，采集写入缓冲区的排列顺序，按数字从小到大排序。

- `scan_elements/in_*_type`：数据在缓冲区中的存储方式，其格式为：`[be|le]:[s|u]bits/storagebits[Xrepeat][>>shift]`

1.2.3 IIO trigger

IIO 框架提供了触发器机制，使得硬件中断的使能与否可由用户自主控制。驱动通过`devm_iio_trigger_alloc`分配触发器，在`trigger->ops`字段定义回调，并通过`devm_iio_trigger_register`注册至 IIO 框架。驱动初始化时，在 sysfs 目录下创建如下文件：

- `/sys/bus/iio/devices/triggerY/*`：触发器注册成功后，内核为其自动分配编号 Y 并在此目录下创建属性文件，可通过`name`属性查询触发器名称。
- `/sys/bus/iio/devices/iio:deviceX/trigger/*`：需`devm_iio_triggered_buffer_setup`使能触发器缓冲区后创建，提供配置文件`current_trigger`更改当前触发器名称。

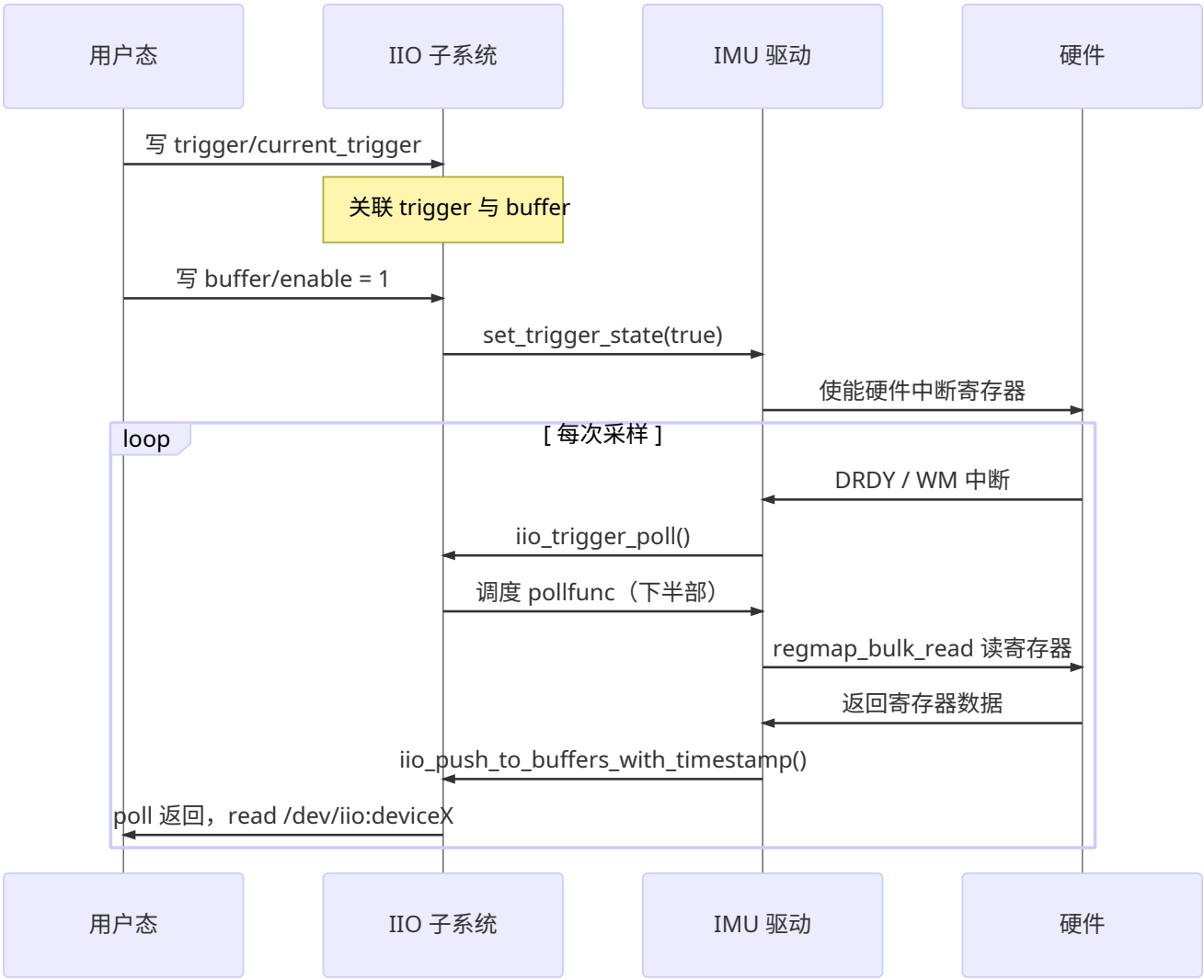
1.2.3.1 trigger 回调

`set_trigger_state`回调负责开启或者关闭触发器，当用户空间通过 sysfs 使能`buffer/enable`（见 中断使能）时，IIO 框架会执行此回调函数。

```
struct iio_trigger_ops {
    // 设置触发器状态
    int (*set_trigger_state)(struct iio_trigger *trig, bool state);
};
```

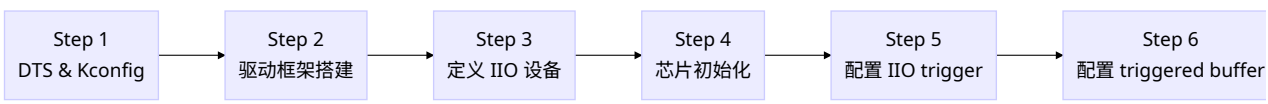
1.2.4 triggered-buffer

由上述内容可知，IIO 触发器与缓冲区需紧密配合。从实现角度看，IMU 驱动通常使用触发器使能硬件中断，在硬件中断上半部打印时间戳，在下半部 / 进程上下文采集传感器数据，而后把数据推送至缓冲区内。整条链路流程如下图所示。



1.3 驱动适配 IIO 框架流程

本小节主要针对IMU驱动进行开发指导。适配一款新 IMU 芯片时，可按以下 6 个步骤依序推进。



1.3.1 DTS & Kconfig

内核通过 DTS 完成总线地址、引脚复用与中断配置，所有参数需参照具体硬件电路设计。

1.3.1.1 总线配置

IMU 通过 I2C 或 SPI 与 SoC 通信，以 I2C 为例：

```
&i2c4 {
    status = "okay";
    clock-frequency = <400000>;          // 总线频率，最高可配置至芯片上限
    imu@68 {
        compatible = "vendor,xxx-imu";   // 与驱动 of_device_id 保持一致
        reg = <0x68>;                    // I2C 设备地址，由硬件决定
        status = "okay";
    };
};
```

- `clock-frequency`：总线速率影响寄存器读写时延，建议配置为芯片支持的最大值。SPI 同理，在 `&spiX { max-freq = <...>; }` 中设置。

1.3.1.2 中断 / 引脚配置

IMU 通过硬件中断触发采样，需在 DTS 中声明中断引脚及 pinctrl 复用。

```
&i2c4 {
    imu@68 {
        pinctrl-names = "default";
        pinctrl-0 = <&xxx_int>;                       // 使用的 pinctrl 配置组
        int1-gpio = <&gpio0 RK_PA4 GPIO_ACTIVE_HIGH>; // INT1 引脚，默认高电平
        interrupt-parent = <&gpio0>;                  // 所属 GPIO 控制器
        interrupts = <4 IRQ_TYPE_EDGE_FALLING>;       // PA4，下降沿触发
    };
};

&pinctrl {
    imu {
        xxx_int: xxx-int {
            // PA4 配置为 GPIO 功能，上拉输入
            rockchip,pins = <0 RK_PA4 RK_FUNC_GPIO &pcfg_pull_up>;
        };
    };
};
```

- `GPIO_ACTIVE_HIGH` + `IRQ_TYPE_EDGE_FALLING`：INT1 默认输出高电平，中断信号为下降沿，两者须与硬件原理图一致。
- 配置完成后可通过 `/sys/kernel/debug/pinctrl/` 确认引脚状态。

1.3.1.3 Kconfig 配置

在驱动目录下添加 `Kconfig` 和 `Makefile`，将驱动纳入内核编译体系：

```
config IMU_xxx
    tristate
    select IIO_BUFFER
    select IIO_TRIGGERED_BUFFER

config IMU_xxx_I2C
    tristate "xxx IMU I2C driver"
    depends on I2C
    select IMU_xxx
    select REGMAP_I2C

config IMU_xxx_SPI
    tristate "xxx IMU SPI driver"
    depends on SPI_MASTER
    select IMU_xxx
    select REGMAP_SPI
```

1.3.2 驱动框架搭建

在 `drivers/iio/imu/` 下建立驱动目录，根据底层通信总线可选创建 `xxx_i2c.c` / `xxx_spi.c`，声明 `of_device_id` 匹配 DTS `compatible`，注册总线驱动入口；根据 IMU 寄存器表单配置 `regmap_config`，使核心层通过统一 API 读写寄存器，无需关心底层总线细节。

```
static const struct of_device_id xxx_of_match[] = {
    { .compatible = "vendor,xxx-imu" }, // 与 DTS 匹配
    {}
};

static const struct regmap_config xxx_regmap_config = {
    .reg_bits     = 8,        // 寄存器地址位宽
    .val_bits     = 8,        // 寄存器值位宽
    .max_register = 0x7F,     // 最高有效寄存器地址
};

static int xxx_i2c_probe(struct i2c_client *client)
{
    struct regmap *regmap = devm_regmap_init_i2c(client, &xxx_regmap_config);
    return xxx_core_probe(regmap, client->irq, client->name);
}

static struct i2c_driver xxx_i2c_driver = {
    .driver = {
        .name           = "xxx-imu",
        .of_match_table = xxx_of_match,
    },
    .probe = xxx_i2c_probe,
```

```
    .remove = xxx_i2c_remove,
};
module_i2c_driver(xxx_i2c_driver);
```

SPI 总线侧的实现与 I2C 类似，设备树匹配表及 regmap 配置保持一致。

```
static int xxx_spi_probe(struct spi_device *spi)
{
    const struct spi_device_id *id = spi_get_device_id(spi);
    struct regmap *regmap = devm_regmap_init_spi(spi, &xxx_regmap_config);
    return xxx_core_probe(regmap, spi->irq, id->name);
}

static struct spi_driver xxx_spi_driver = {
    .driver = {
        .name           = "xxx-imu",
        .of_match_table = xxx_of_match,
    },
    .probe = xxx_spi_probe,
    .remove = xxx_spi_remove,
};
module_spi_driver(xxx_spi_driver);
```

在 `xxx_core.c` 中实现核心 `probe`，分配并注册 IIO 设备：

```
int xxx_core_probe(struct regmap *regmap, int irq, const char *name)
{
    // IIO 设备初始化
    struct iio_dev *iio_dev = devm_iio_device_alloc(dev, sizeof(*data));
    iio_dev->name  = name;
    iio_dev->modes = INDIO_BUFFER_TRIGGERED;

    return devm_iio_device_register(dev, iio_dev);
}
```

1.3.3 定义 IIO 设备通道与属性

在 `xxx_core.c` 声明 IMU 传感器通道以及时间戳通道，通常包括三轴加速计和三轴陀螺仪。

```
#define IIO_CHANNEL(_type, _axis, _index) {                 \
    .type = _type,                                          \
    .modified = 1,                                          \
    .channel2 = _axis,                                      \
    .info_mask_separate = BIT(IIO_CHAN_INFO_RAW),           \
    .info_mask_shared_by_type = BIT(IIO_CHAN_INFO_SCALE) |  \
        BIT(IIO_CHAN_INFO_SAMP_FREQ),                       \
    .scan_index = _index,                                   \
    .scan_type = {                                          \
        .sign       = 's',                                  \
        .realbits   = 16,                                   \
```

```
        .storagebits = 16,                                      \
        .endianness  = IIO_BE,                                  \
    },                                                          \
}

static const struct iio_chan_spec xxx_channels[] = {
    // 三轴加速计
    IIO_CHANNEL(IIO_ACCEL,    IIO_MOD_X, 0),
    IIO_CHANNEL(IIO_ACCEL,    IIO_MOD_Y, 1),
    IIO_CHANNEL(IIO_ACCEL,    IIO_MOD_Z, 2),
    // 三轴陀螺仪
    IIO_CHANNEL(IIO_ANGL_VEL, IIO_MOD_X, 3),
    IIO_CHANNEL(IIO_ANGL_VEL, IIO_MOD_Y, 4),
    IIO_CHANNEL(IIO_ANGL_VEL, IIO_MOD_Z, 5),
    // 温度
    IIO_CHANNEL(IIO_TEMP,     IIO_NO_MOD, 6),
    IIO_CHAN_SOFT_TIMESTAMP(7),
};
```

在 `xxx_core.c` 声明 IIO 设备属性。读写函数可通过已封装的 regmap 接口进行总线通信。

```
static int xxx_read_raw(struct iio_dev *indio_dev,
                        struct iio_chan_spec const *chan,
                        int *val, int *val2, long mask);
static int xxx_write_raw(struct iio_dev *indio_dev,
                         struct iio_chan_spec const *chan,
                         int val, int val2, long mask);
static const struct attribute_group xxx_attrs_group = {
    .attrs = xxx_attrs,
};

static const struct iio_info xxx_info = {
    .attrs = &xxx_attrs_group,
    .read_raw  = xxx_read_raw,
    .write_raw = xxx_write_raw,
};
```

在 `xxx_core_probe` 中挂载：

```
int xxx_core_probe(struct regmap *regmap, int irq, const char *name)
{
    iio_dev->channels  = xxx_channels;
    iio_dev->info  = &xxx_info;
}
```

1.3.4 芯片初始化实现

驱动开发者需根据芯片厂商提供的 FS / Datasheet 手册实现，步骤包括但不限于传感器复位，初始化陀螺仪、加速计等传感器，初始化时钟、电源和中断等硬件资源。

1.3.5 配置 IIO trigger

```
static int xxx_set_trigger_state(struct iio_trigger *trig, bool state)
{
    // 具体实现：开启 Data Ready / Watermark 中断
}

static const struct iio_trigger_ops xxx_trigger_ops = {
    .set_trigger_state = xxx_set_trigger_state,
};

// 注册 IIO trigger
int xxx_core_probe(struct regmap *regmap, int irq, const char *name)
{
    trig = devm_iio_trigger_alloc(dev, "%s-dev%d",
                                  iio_dev->name, iio_device_id(iio_dev));
    trig->ops = &xxx_trigger_ops;
    iio_trigger_set_drvdata(trig, iio_dev);
    devm_iio_trigger_register(dev, trig);
}
```

1.3.6 配置 triggered-buffer

如triggered-buffer配置所述，trigger 负责使能硬件中断，而中断处理、传感器信号采集及数据推送至缓冲区均需在驱动中实现。本节说明 ISR 的设计方法及 buffer 的初始化与配置流程。

buffer 及中断配置如下，推荐使用 `devm_request_irq` 申请硬件中断，通过 `iio_trigger_generic_data_rdy_poll` 触发采样。采样分为两步，第一步进入 `iio_pollfunc_store_time` 记录时间戳，第二步在 `xxx_handler` 中采集传感器数据并推送至缓冲区。

```
int xxx_core_probe(struct regmap *regmap, int irq, const char *name)
{
    devm_iio_triggered_buffer_setup(dev, indio_dev,
                                    iio_pollfunc_store_time, xxx_handler, NULL);
    devm_request_irq(&iio_dev->dev, irq,
                     &iio_trigger_generic_data_rdy_poll,
                     irq_type, name, trig);
}
```

具体采集的传感器通道及寄存器地址范围应根据芯片设计与应用需求确定；本节仅提供驱动框架示例。

```
static irqreturn_t xxx_handler(int irq, void *p)
{
    // iio_pollfunc_store_time已打时间戳，下半部直接读取
    struct iio_poll_func *pf = p;
    s64 data_timestamp = pf->timestamp;

    // 采集数据至 sensor_data
    u8 sensor_data[13];
```

```
    unsigned int val;
    regmap_bulk_read(regmap, 0x01, sensor_data, 12);    // 从0x01开始连续读12个reg
    regmap_read(regmap, 0x3A, &val);
    sensor_data[12] = (u8)val;                          // 读取0x3A寄存器

    // 推送至缓冲区
    iio_push_to_buffers_with_timestamp(iio_dev, sensor_data, data_timestamp);
    return IRQ_HANDLED;
}
```

2. IMU 使用指南

本章介绍通过 Linux IIO 接口使用传感器的方法，包括 sysfs 参数配置、中断采集使能与数据读取。

2.1 IIO sysfs 配置

IIO 驱动通过 sysfs 暴露传感器参数，本节介绍用户空间配置 IIO sysfs 文件修改传感器参数及使能 trigger 的方法。

2.1.1 采样频率

采样频率（ODR）通过 `in_{type}_sampling_frequency` 文件读写，可选值列表由 `_available` 后缀文件提供。

```
# 查看支持的采样频率（单位 Hz）
cat /sys/bus/iio/devices/iio:deviceX/in_accel_sampling_frequency_available

# 设置采样频率
echo 400 > /sys/bus/iio/devices/iio:deviceX/in_accel_sampling_frequency

# 写入不在列表中的值将返回 -EINVAL
echo 9 > /sys/bus/iio/devices/iio:deviceX/in_accel_sampling_frequency
-bash: echo: write error: Invalid argument
```

加速计与陀螺仪的 ODR 文件相互独立，分别为 `in_accel_sampling_frequency` 和 `in_anglvel_sampling_frequency`。

2.1.2 量程与精度

量程通过 `in_{type}_scale` 文件设置，单位为 SI 物理量每 LSB。精度与量程负相关：

$$scale*{accel} = *frac{range*{g} \times 2 \times 9.8}{2^{bits}}\quad(m/s^2/LSB)$$

$$scale*{gyro} = *frac{range*{dps} \times 2 \times \pi}{2^{bits} \times 180}\quad(rad/s/LSB)$$

```
# 查看加速计可选精度（单位 m/s²/LSB）
cat /sys/bus/iio/devices/iio:deviceX/in_accel_scale_available

# 设置加速计量程
echo 0.000598 > /sys/bus/iio/devices/iio:deviceX/in_accel_scale

# 查看陀螺仪可选精度（单位 rad/s/LSB）
cat /sys/bus/iio/devices/iio:deviceX/in_anglvel_scale_available

# 设置陀螺仪量程
echo 0.000133231 > /sys/bus/iio/devices/iio:deviceX/in_anglvel_scale
```

量程选择建议：

- **大量程** — 可捕捉快速/剧烈运动不饱和，适合**无人机、运动相机、手持云台**等运动剧烈的场景。
- **小量程** — 分辨率高、量化噪声小，但快速运动时易截断，适合**机器人底盘、AGV、静态姿态估计**等运动平缓的场景。

2.1.3 低通滤波

LPF 截止频率通过 `in_{type}_lpf_bw` 文件设置（部分驱动使用 `filter_low_pass_3db_frequency`），写 0 表示 bypass。

```
# 查看可选截止频率（单位 Hz）
cat /sys/bus/iio/devices/iio:deviceX/in_accel_lpf_bw_available

# 设置为 180 Hz（拦截高于 180 Hz 的信号）
echo 180 > /sys/bus/iio/devices/iio:deviceX/in_accel_lpf_bw

# 关闭滤波器（bypass）
echo 0 > /sys/bus/iio/devices/iio:deviceX/in_accel_lpf_bw
```

LPF 带宽应与 ODR 和量程配合：高动态场景提高采样率并放宽 LPF；低动态场景降低采样率并收窄 LPF 以抑制高频噪声。

2.1.4 中断使能

IIO 驱动通过 trigger + buffer 机制实现中断驱动采集，使能步骤与具体芯片无关。

```
# 查找 trigger id 与名称
ls /sys/bus/iio/devices/ | grep trigger
# triggerX

cat /sys/bus/iio/devices/triggerX/name
# <trigger_name>

# 查找对应 IIO 设备 id
cat /sys/bus/iio/devices/iio:deviceX/name
```

```
cd /sys/bus/iio/devices/iio:deviceX

# 绑定 trigger
echo "<trigger_name>" > trigger/current_trigger

# 使能需采集的通道
echo 1 > ./scan_elements/in_accel_x_en
echo 1 > ./scan_elements/in_accel_y_en
echo 1 > ./scan_elements/in_accel_z_en
echo 1 > ./scan_elements/in_anglvel_x_en
echo 1 > ./scan_elements/in_anglvel_y_en
echo 1 > ./scan_elements/in_anglvel_z_en
echo 1 > ./scan_elements/in_temp_en
echo 1 > ./scan_elements/in_timestamp_en

# 配置 kfifo buffer
echo 128 > ./buffer/length
echo 1 > ./buffer/watermark

# 使能中断触发采集
echo 1 > ./buffer/enable

# 关闭采集
echo 0 > ./buffer/enable
```

使能后驱动会调用 `trigger_set_state(true)` 使能硬件中断；关闭时调用 `trigger_set_state(false)` 关闭中断。数据通过 `/dev/iio:deviceX` 以 `read()` 读取。

2.2 数据读取与处理

本小节说明通过 Linux IIO 接口读取加速计/陀螺仪数据的方法。

2.2.1 IIO sysfs read

```
# 获取三轴加速计传感器数据
cat /sys/bus/iio/devices/iio:deviceX/in_accel_x_raw
cat /sys/bus/iio/devices/iio:deviceX/in_accel_y_raw
cat /sys/bus/iio/devices/iio:deviceX/in_accel_z_raw

# 获取三轴陀螺仪传感器数据
cat /sys/bus/iio/devices/iio:deviceX/in_anglvel_x_raw
cat /sys/bus/iio/devices/iio:deviceX/in_anglvel_y_raw
cat /sys/bus/iio/devices/iio:deviceX/in_anglvel_z_raw

# 获取温度传感器数据
cat /sys/bus/iio/devices/iio:deviceX/in_temp_x_raw
```

获取原始加速计/陀螺仪数据需要与精度单位相乘，若加速计量程为 $0.000598m/s^2$，静止时 z 轴加速计原始值为 16412，则 z 轴加速度为 $16412×0.000598 = 9.814376 m/s^2$。

> **注意：** IIO 框架输出的陀螺仪角速度单位为 **rad/s（弧度/秒）**，而非度/秒（°/s）。若应用层按 °/s 解读数据，实际值将偏大约 57 倍，导致姿态解算异常。使用前请确认单位。

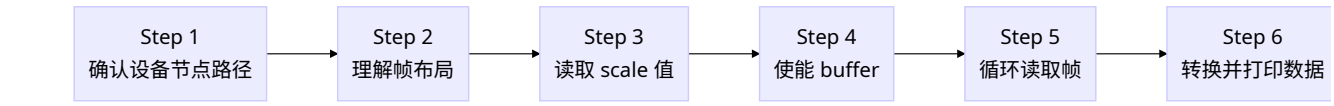
温度传感器数据需根据寄存器值进行转换，转换公式根据 packet 中温度传感器数据大小决定：

`8-bit quantity(FIFO):` $$Celsius = \frac{TEMP_{DATA}}{2} + 25$$

`16-bit quantity(DRDY):` $$Celsius = \frac{TEMP_{DATA}}{128} + 25$$

2.2.2 IIO buffer 读取流程

用户空间通过 `read` / `poll` 系统调用从 `/dev/iio:deviceX` 读取 IMU 数据，整体流程分为 6 步。



2.2.2.1 确认设备节点路径

IIO 框架会为每个注册的 IMU 设备创建两类访问入口：

- **字符设备** `/dev/iio:deviceX`：用于 `read()` 批量读取采样帧
- **sysfs 节点** `/sys/bus/iio/devices/iio:deviceX/`：用于读取传感器参数（scale、ODR 等）

```
# 查看当前系统有多少个 IIO 设备
ls /sys/bus/iio/devices/

# 查看设备名（对应驱动中 iio_dev->name）
cat /sys/bus/iio/devices/iio:device1/name
```

驱动开发者需保证 DTS 中 `compatible` 与驱动匹配，系统启动后设备节点才会出现。

2.2.2.2 理解帧布局

从 `/dev/iio:deviceX` 读取的数据帧由 **scan elements（通道数据）+ timestamp** 组成，具体布局由驱动层的 `iio_chan_spec.scan_type` 决定。例如，ICM42670 驱动定义的帧格式如下：

偏移	字段	类型	说明
0	ax	s16 BE	加速度 X 轴原始值
2	ay	s16 BE	加速度 Y 轴原始值
4	az	s16 BE	加速度 Z 轴原始值
6	gx	s16 BE	角速度 X 轴原始值
8	gy	s16 BE	角速度 Y 轴原始值
10	gz	s16 BE	角速度 Z 轴原始值
12	temp	s8/s16	温度原始值
16	timestamp	s64 LE	时间戳（ns）

```
struct imu_frame {
    int16_t  ax, ay, az;   /* accel raw, BE */
    int16_t  gx, gy, gz;   /* gyro  raw, BE */
    int16_t  temp;         /* temp  raw, 16-bit */
    uint8_t  pad[2];       /* padding to 8B margin */
    int64_t  timestamp;    /* ns, LE */
} __attribute__((packed));
```

2.2.2.3 读取 scale 值

`read_raw` 返回的是传感器寄存器原始值，需要乘以 `scale` 才能得到物理量。scale 通过 sysfs 文件读取：

```
float accel_scale = read_sysfs_float("/sys/bus/iio/devices/iio:device1/in_accel_scale");
float gyro_scale  = read_sysfs_float("/sys/bus/iio/devices/iio:device1/in_anglvel_scale");
```

这两个文件对应驱动中定义的 `info_mask_shared_by_type = BIT(IIO_CHAN_INFO_SCALE)`。若文件不存在或读取失败，返回默认值 `0.0f`，数据将全为 0。

2.2.2.4 使能 buffer

用户空间读取数据前，需先通过 sysfs 使能 buffer，配置请参考中断使能。使能 buffer 后，驱动侧的 `xxx_handler` 开始收到中断，采集的数据被推送至 `kfifo`，用户空间 `read()` 才有数据可读。

2.2.2.5 循环读取帧

打开字符设备，以阻塞 `read()` 循环采集数据：

```
int fd = open("/dev/iio:device1", O_RDONLY);
if (fd < 0) {
    perror("open");
```

```
    return 1;
}

struct imu_frame frame;
while (running) {
    ssize_t n = read(fd, &frame, sizeof(frame));
    if (n < 0) { perror("read"); break; }
    if ((size_t)n != sizeof(frame)) {
        fprintf(stderr, "short read: %zd (expected %zu)\n", n, sizeof(frame));
        continue;
    }
    // 处理 frame ...
}
```

`read()` 每次返回一帧完整数据（24 字节）。若驱动 buffer length 配置大于 1，`read()` 可能一次返回多帧，此时返回值 `n` 将为 `sizeof(frame)` 的整数倍。

2.2.2.6 转换并打印数据

驱动侧的 `scan_type.endianness` 决定了数据的存储字节序，若为大端序，用户空间读取后需通过 `ntohs()` 转换为主机字节序。timestamp 固定为小端序。将原始值转换为物理量并打印：

```
#include <arpa/inet.h>   // ntohs

// 大端→主机字节序
int16_t ax = ntohs(frame.ax);
int16_t ay = ntohs(frame.ay);
int16_t az = ntohs(frame.az);

// 原始值 × scale = 物理量
float ax_phys = ax * accel_scale;   // m/s²
float gx_phys = gx * gyro_scale;    // rad/s

// 计算采样间隔
float dt_ms = (float)(frame.timestamp - prev_ts) / 1e6f;
prev_ts = frame.timestamp;

printf("a(m/s2) %6.3f %6.3f %6.3f | g(rad/s) %7.4f %7.4f %7.4f\n",
       ax_phys, ay_phys, az_phys, gx_phys, gy_phys, gz_phys);
```

3. 数据分析与标定

本章介绍 IMU 数据质量判断与校准标定方法。

3.1 数据校准与标定

EIS 与 SLAM 场景均需要进行标定。EIS 场景主要关注 IMU 与图像旋转方向的一致性标定，SLAM 场景需要标定 IMU 与相机之间的外参和 IMU 噪声参数。以下主要介绍 SLAM 的外参标定方法。EIS 标定方法可以参考相关文档 `Rockchip_Developer_Guide_EIS_CN.pdf`（位于 `docs/cn/Socs/RV1126B/RKIPC/` 路径下）。

3.1.1 了解标定参数

SLAM 算法（以 ORB-SLAM3 为例）融合 IMU 数据需要以下参数：

参数名称	含义	单位
`T_b_c`	相机坐标系到 IMU 坐标系的转换矩阵（4×4）	—
`NoiseGyro`	陀螺仪随机噪声密度	rad/s/√Hz
`NoiseAcc`	加速度计随机噪声密度	$m/s^2/\sqrt{Hz}$
`GyroWalk`	陀螺仪零偏随机游走	$rad/s^2/\sqrt{Hz}$
`AccWalk`	加速度计零偏随机游走	$m/s^3/\sqrt{Hz}$
`Frequency`	IMU 数据发布频率	Hz

其中 `T_b_c` 需通过采集数据进行标定，噪声参数可查阅芯片 datasheet 所给的 Allan 方差曲线，也可通过 kalibr 的 `kalibr_calibrate_imu_camera` 命令一并输出。

3.1.2 安装标定工具

```
# 拉取 kalibr ROS2 版本源码
git clone https://github.com/yuhang0131/kalibr_ros2.git
cd kalibr_ros2

# 安装系统依赖
sudo apt install -y \
  libeigen3-dev libboost-all-dev libopencv-dev \
  libsuitesparse-dev libtbb-dev \
  python3-numpy python3-matplotlib python3-scipy python3-opencv
```

```
# 安装 Python 依赖
pip install python-igraph

# 编译（如报缺少依赖，source 后重新执行编译）
./build_workspace.sh
source install/setup.bash
```

3.1.3 采集标定数据

采集前需确认相机与 IMU 的时间戳已同步（硬件同步或软件对时），否则标定结果的时延项将不可信。棋盘格靶标录制需遵从以下要求。

棋盘格本身要求

- 平整硬质板，不能卷曲或反光。
- 格子尺寸、角点数量（内角点数）要已知且准确，和 `target.yaml` 参数一致。
- 黑白对比清晰，边缘不糊、不脏污。

运动要求

- 将相机与 IMU **全程保持刚性固连**，对着棋盘格靶标录制约 120 s 的 bag 数据。
- 录制期间按顺序执行以下平移+转动+8字轨迹运动。距靶标 1～2 m，静止保持 10 s（用于估计初始零偏）；前后、左右、上下平移运动 20 s；俯仰、偏航、翻滚转动各方向 20 s；8 字轨迹 + 随机运动 30 s，保持运动连贯、激励充分；改变距离与角度观察靶标 30 s；最后挪回原位静止 10 s。

画面采集要求

- 棋盘格在图像中不要太小，建议至少占画面 1/4 到 1/2。
- 棋盘格不能驻留某区域画面过久，要覆盖中心+四角+边缘区域，保证不同距离和不同倾角（俯仰/偏航/滚转）。
- 避免强反光、频闪灯、运动模糊和严重过曝。

```
# 同时录制 RGB 图像与 IMU 数据，时长约 120 s
timeout 120s ros2 bag record -o imu_cam_calibrate \
  /your_cam_topic/color/image_rgb \
  /your_imu_topic/data
```

3.1.4 创建配置文件

camchain.yaml（相机内参，由结构光模组出厂标定参数填入）

```
cam0:
  camera_model: pinhole
  intrinsics: [fx, fy, cx, cy]         # 填入实际内参
  distortion_model: radtan
  distortion_coeffs: [k1, k2, p1, p2] # 已出厂矫正则填 [0,0,0,0]
  resolution: [width, height]
  rostopic: /your_cam_topic/color/image_rgb
```

imu.yaml（IMU 噪声参数，参考 datasheet；如不确定可先用以下经验值）

```
accelerometer_noise_density: 0.006      # m/s^2/sqrt(Hz)
accelerometer_random_walk:   0.0002     # m/s^3/sqrt(Hz)
gyroscope_noise_density:     0.0004     # rad/s/sqrt(Hz)
gyroscope_random_walk:       4.0e-06    # rad/s^2/sqrt(Hz)
update_rate: 200.0                      # Hz，与实际 IMU ODR 保持一致
rostopic: /your_imu_topic/data
```

target.yaml（棋盘格靶标参数，按实物填写）

```
target_type: 'checkerboard'
targetCols: 8            # 内角点列数
targetRows: 11           # 内角点行数
rowSpacingMeters: 0.05   # 行方向角点间距 (m)
colSpacingMeters: 0.05   # 列方向角点间距 (m)
```

3.1.5 执行标定

```
cd kalibr_ros2 && source install/local_setup.bash

ros2 run kalibr_imu_camera kalibr_calibrate_imu_camera \
  --bag  imu_cam_calibrate \
  --cams camchain.yaml \
  --imu  imu.yaml \
  --target target.yaml \
  --show-extraction
```

标定完成后，kalibr 会在当前目录输出 `*-results-imucam.txt` 及 `*-camchain.yaml`，输出结果如下图所示，其中包含：

- `T_ci / T_ic`：相机相对 IMU / IMU 相对相机的外参矩阵（即 `T_b_c`）
- `timeshift_cam0_to_imu0`：相机相对于 IMU 的时间偏移（s）
- 各噪声参数的拟合估计值

```
Calibration results
====================
Normalized Residuals
----------------------------
Reprojection error (cam0):     mean 0.09968181195443296, median 0.0903287683591986, std: 0.060634278731619765
Gyroscope error (imu0):        mean 0.3250498564274567, median 0.25135580409887376, std: 0.8993709899513046
Accelerometer error (imu0):    mean 0.292109487199641, median 0.1980429974587293, std: 0.9634169383777025

Residuals
----------------------------
Reprojection error (cam0) [px]:     mean 0.09968181195443296, median 0.0903287683591986, std:
0.060634278731619765
Gyroscope error (imu0) [rad/s]:     mean 0.0018387596616285466, median 0.0014218831485512883, std:
0.0050876106063762055
Accelerometer error (imu0) [m/s^2]: mean 0.024786311909734942, median 0.01680450557634932, std:
0.08174863802842267

Transformation (cam0):
-----------------------
T_ci:  (imu0 to cam0):
[[-0.99974638  0.0038015   0.02219753 -0.00044106]
 [-0.00412274 -0.99988717 -0.01444453 -0.0080838 ]
 [ 0.02214011 -0.01453238  0.99964925 -0.00744897]
 [ 0.         0.         0.         1.        ]]

T_ic:  (cam0 to imu0):
[[-0.99974638 -0.00412274  0.02214011 -0.00030935]
 [ 0.0038015  -0.99988717 -0.01453238 -0.00818946]
 [ 0.02219753 -0.01444453  0.99964925  0.00733938]
 [ 0.         0.         0.         1.        ]]

timeshift cam0 to imu0: [s] (t_imu = t_cam + shift)
-0.00170162896166359


Gravity vector in target coords: [m/s^2]
[ 9.66151343  0.20447352 -1.66786444]
```

3.1.6 将标定结果写入算法配置

将上述输出填入 ORB-SLAM3 的 yaml 配置文件：

```
IMU.T_b_c1: !!opencv-matrix
   rows: 4
   cols: 4
   dt: f
   data: [ r00, r01, r02, t0,
           r10, r11, r12, t1,
           r20, r21, r22, t2,
           0.,  0.,  0.,  1. ]   # 由 T_imu_cam 填入

IMU.NoiseGyro:  0.0004    # 由标定结果或 datasheet 填入
IMU.NoiseAcc:   0.006
IMU.GyroWalk:   4.0e-06
IMU.AccWalk:    0.0002
IMU.Frequency:  200       # 与 imu.yaml 中 update_rate 保持一致
```

3.2 判断数据是否正常

本小节说明如何判断 IMU 数据正常与否。用户可通过脚本解析 IMU 采样数据，从 10 个维度对输入数据的准确性进行检验。输入的 IMU 数据需遵从每行 8 列的格式，列顺序固定为：

```
accel_x  accel_y  accel_z  angle_x  angle_y  angle_z  temp  timestamp
```

其中加速度单位 m/s²、角速度单位 rad/s、温度为原始寄存器值或摄氏度（由脚本顶部 `TEMP_MODE` 决定）、时间戳单位为纳秒（ns）。

3.2.1 采集数据

在开发板上通过 IIO sysfs 循环读取并将结果追加写入日志文件，示例脚本如下：

```
#!/bin/sh

DEV=/sys/bus/iio/devices/iio:device1
OUT=./imu.log

# 读取一次 scale，后续直接换算为物理量
A_SCALE=$(cat $DEV/in_accel_scale)
G_SCALE=$(cat $DEV/in_anglvel_scale)

: > $OUT
for i in $(seq 1 2000); do
    AX=$(cat $DEV/in_accel_x_raw)
    AY=$(cat $DEV/in_accel_y_raw)
    AZ=$(cat $DEV/in_accel_z_raw)
    GX=$(cat $DEV/in_anglvel_x_raw)
    GY=$(cat $DEV/in_anglvel_y_raw)
    GZ=$(cat $DEV/in_anglvel_z_raw)
    T=$(cat $DEV/in_temp_x_raw)
    TS=$(date +%s%N)
    awk -v ax=$AX -v ay=$AY -v az=$AZ \
        -v gx=$GX -v gy=$GY -v gz=$GZ \
        -v t=$T -v ts=$TS \
        -v as=$A_SCALE -v gs=$G_SCALE \
        'BEGIN{printf "%.6f %.6f %.6f %.6f %.6f %.6f %d %s\n",
               ax*as, ay*as, az*as, gx*gs, gy*gs, gz*gs, t, ts}' >> $OUT
done
```

实际使用中建议通过使能 IIO trigger ，经 IIO buffer 采集以获得稳定的高频数据，上述 shell 脚本轮询仅用于快速验证。

3.2.2 数据检查

数据检查脚本 `visualize_imu.py` 存放至网盘，通过本链接下载。将 IMU 采样日志 `imu.log` 拷贝至 PC，通过脚本执行：

```
python visualize_imu.py imu.log
```

脚本会在同目录输出 `imu.log.png`，图中包含 11 个子图：加速度/角度原始曲线与积分、温度、时间戳间隔与抖动、分布直方图、异常标注及统计汇总。脚本内置 `AnomalyDetector`，从以下维度检查数据，任一项计数大于 0 即为异常：

检测项	判据	常见原因
`duplicate_timestamp`	相邻时间戳完全相同	采集线程被抢占、sysfs 读取速率过慢
`timestamp_jump`	相邻间隔出现 < -1 ms 的倒流	系统时钟被修改、多线程乱序写入
`timestamp_gap`	相邻间隔偏离均值超过 `TIMESTAMP_JITTER_THRESHOLD`（默认 100 μs）	中断延迟、I2C 总线阻塞
`accel_out_of_range`	加速计超量程	量程配置过小导致饱和，或机械冲击
`angle_out_of_range`	陀螺仪超量程	陀螺仪量程过小、数据解析错误
`temp_out_of_range`	温度传感器超量程	温度寄存器未正确换算
`accel_outlier` / `angle_outlier`	Z-score > 3 或超出 Q3 ± 3·IQR	单点抖动、总线误码
`gyro_rate_outlier`	相邻角速度跳变 > 5 rad/s	采样间隔不稳、数据错位
`stuck_value`	连续 ≥ 5 个样本完全相同	驱动未更新、FIFO 卡死
`invalid_data`	出现 `NaN` / `Inf`	原始寄存器读取失败

终端同时会打印统计汇总，图中第 11 块面板也会列出关键异常计数与各通道 `mean ± std`。**判断数据是否正常的流程：**

- 查看总样本数与实际采样率是否与配置的 ODR 一致，若偏差显著应先排查采集流程。
- 若 `duplicate_timestamp` / `timestamp_gap` 非零，说明采集链路存在抖动，需优先解决而非怀疑传感器。
- 若 `accel_out_of_range` 或 `angle_out_of_range` 非零，检查当前量程是否与运动幅度匹配，必要时调大量程（参考 量程与精度配置）。
- 若 `stuck_value` / `invalid_data` 非零，通常为驱动或总线异常，结合 `dmesg` 进一步定位。
- 静置状态下，三轴加速度模长应接近 $9.8\,\text{m/s}^2$，三轴陀螺仪均值应接近 0；偏差过大则需执行 数据校准与标定。

4. 常见问题排查

本章主要说明 IMU 驱动在调试过程中的常见问题及对应排查方法。

4.1 PCB 布板注意事项

- 退耦电容（100 nF + 10 μF）尽量靠近电源引脚，减小电源域 EMI
- 远离高频电路（Wi-Fi 天线、开关电源）、机械振动源（风扇、马达）
- 安装螺柱/胶层的共振频率若在 ODR 频段内，会在频谱上出现固定峰
- 更完整的布板规范参考：TDK《MEMS Products PCB Design, Mounting, and Handling Guidelines》、Bosch 《BMI08x Design Guide》

4.2 零偏与温漂是否需要工厂矫正

- **EIS 场景**：应用层实时补偿（静置平均）即可，无需工厂校正；但要确保驱动 `calibbias` 属性已对外暴露
- **SLAM/导航场景**：建议工厂做三轴六面法零偏校正，并对全温范围（-20 ~ 60°C）拟合线性温补系数，写入 NVM 或在用户空间启动时加载
- 具体校正方法参考各芯片 datasheet `Self-Test and OTP` 章节

4.3 内置 LPF 对数据的影响

- LPF 截止频率越低，噪声越小，但会**引入相位延迟**；具体延迟特性需参考原厂 datasheet
- SLAM/VIO 等对相位一致性敏感的算法，需注意内置 LPF 可能导致时间戳与实际运动不对齐，建议结合算法需求评估是否关闭或调整 LPF